AAI-540 ML Solution Design Document

Project: Criteo Advertisement CTR Prediction

Team Info

Project Team Group #: 4
Authors: Prashant Khare, Manu Malla, Laxminag Mamillapalli
Business Name: Advertisement
Publication Date: 19-10-2026

Team Workflows

GitHub Project Link: https://github.com/sentinel1282/Group--4_MLOPs---Criteo_Advertisement_CTR_Prediction-

Asana Board Link: https://app.asana.com/1/1218486826829015/project/1218498458917055/dashboard/1218494107081461

Team Tracker Link: https://app.asana.com/1/1218486826829015/project/1218498458917055/overview/1218494107081457

Codebase: https://github.com/sentinel1282/Group--4_MLOPs---Criteo_Advertisement_CTR_Prediction-

Dataset: Criteo CTR Dataset.zip

Project Scope

Project Background:

Online advertising exchanges must decide, within milliseconds, which of thousands of candidate ads to show a user and how much to bid for that impression. The bid is, in effect, the predicted probability of a click multiplied by the value of a click, so the decision is only as good as the exchange's estimate of whether the user will click. Underestimating click-through rate (CTR) makes advertisers under-bid and lose impressions that would have been profitable;

overestimating it spends budget on impressions that were never going to convert. This project builds a system that predicts the probability a user clicks a display ad, using the Criteo Display Advertising Challenge dataset, and wraps that model in the operational machinery a business needs to trust it: a data lake, a feature store, a gated release pipeline, two serving paths and production monitoring.

The output that matters is a calibrated probability, not a class label. A model can rank impressions in exactly the right order — and score well on AUC — while over-stating every probability, and a bidding system that multiplies those probabilities by a value will then overpay on every bid. This is a supervised binary classification problem: each row is one impression, labelled 1 if the user clicked and 0 otherwise, and the goal is a function that maps the impression's features to an honest estimate of the probability of a click.

Technical Background:

The dataset holds 1 million labelled impressions from seven consecutive days of live Criteo traffic — about of tab-separated text with no header — described by 13 anonymised integer features (I1–I13) and 26 categorical features (C0–C25) that Criteo hashed into 8-character hexadecimal identifiers before release. Because the features are anonymised there is no domain knowledge to lean on; the technical challenges are scale, categorical cardinality that ranges from a handful of levels to millions, pervasive and informative missingness, and a file that is ordered in time but carries no timestamp.

Exploratory analysis corrected one assumption in our original proposal: the label is not heavily imbalanced. Criteo sub-sampled negative examples before release, so the click rate in this data is about 26%, far above live traffic. There is therefore no imbalance worth correcting, and — as Model Training & Evaluation shows — correcting it anyway with class weights damages calibration. Absolute click rates from this data are not comparable with real-world CTR, and the design never presents them as if they were.

The results in this version were measured on a 1,000,000-row extract of the Criteo training data (Criteo_1M_with_nans.csv), kept in file order. Its click rate is 25.5%; integer features are missing 20.9% of the time on average (I12 77%) and categorical features 11.6% (C22 74%); categorical cardinality runs from 3 levels to 321,438 (C3). Missingness carries signal: impressions with I6 missing click 12.3% of the time against 29.3% when it is present, which is why every numeric feature gets an explicit missing flag.

Log loss is the primary metric, matching the original competition, because it penalises confident wrong predictions and rewards honest probabilities. AUC is tracked for ranking quality, normalised entropy tells us whether a model beats simply predicting the average click rate, and expected calibration error (ECE) measures the mis-scaling that mis-prices bids. The model is SageMaker's built-in XGBoost, compared against two deliberately simple benchmarks: a constant predictor of the training click rate and a two-feature L2-regularised logistic regression.

Goals vs Non-Goals:

Goals:

- Predict click probability with normalised entropy below 1.0 on a chronologically held-out test split, beating both benchmarks and the currently approved model.
- Produce well-calibrated probabilities, gated at an ECE of 0.02 before registration and monitored in production against a ceiling of 0.01.
- Build a reproducible path from the raw file to a registered model, re-executable as a single parameterised SageMaker Pipeline, with every number traceable to a data version, a job and a commit.
- Detect degradation before it reaches business metrics, using label-free data-quality signals first and model quality once labels arrive.
- Prevent any unreviewed model from reaching serving: an automated four-condition quality gate followed by manual approval in the Model Registry.

Non-Goals:

- Serving live bidding traffic at production volume. The real-time endpoint validates the serving path and feeds the monitors; low-latency bidding is a future extension.
- Reverse-engineering the anonymised or hashed feature identifiers to recover their real-world meaning.
- Modelling downstream conversion, revenue or bid strategy; click probability is the output, and turning it into a bid belongs to the consuming system.
- Exhaustive hyperparameter search, ensembling or deep architectures beyond what a meaningful model comparison needs.

Solution Overview

The CriteoCTR system takes raw impression logs to calibrated click probabilities through five stages, shown in Figure 1 with every hand-off numbered. Ingest: the Criteo archive (dac.tar.gz) is uploaded to an S3 landing folder and stream-extracted into an immutable S3 raw zone, which Athena catalogues so that whole-dataset questions — row counts, click rate per day, null rates, cardinality — are answered in SQL rather than on a notebook. Prepare: a chronological one-in-45 systematic sample (about 1.02 million rows spanning all seven days in order) is engineered into features, written to a curated Parquet zone and registered in SageMaker Feature Store, with the label in a feature group of its own.

Train and govern: XGBoost is trained and tuned on a chronological split, evaluated once on a locked test split, and registered in the SageMaker Model Registry only if it clears a four-condition quality gate; a person must then approve it. Serve: the approved package scores the held-back production split through Batch Transform, the primary path, and a real-time endpoint with data capture serves the monitors. Monitor: Model Monitor checks data quality and model quality on hourly schedules, CloudWatch carries the dashboard and five alarms, and SNS notifies the team.

The whole path is one parameterised SageMaker Pipeline, so a retrain needs a parameter change, not a code change.

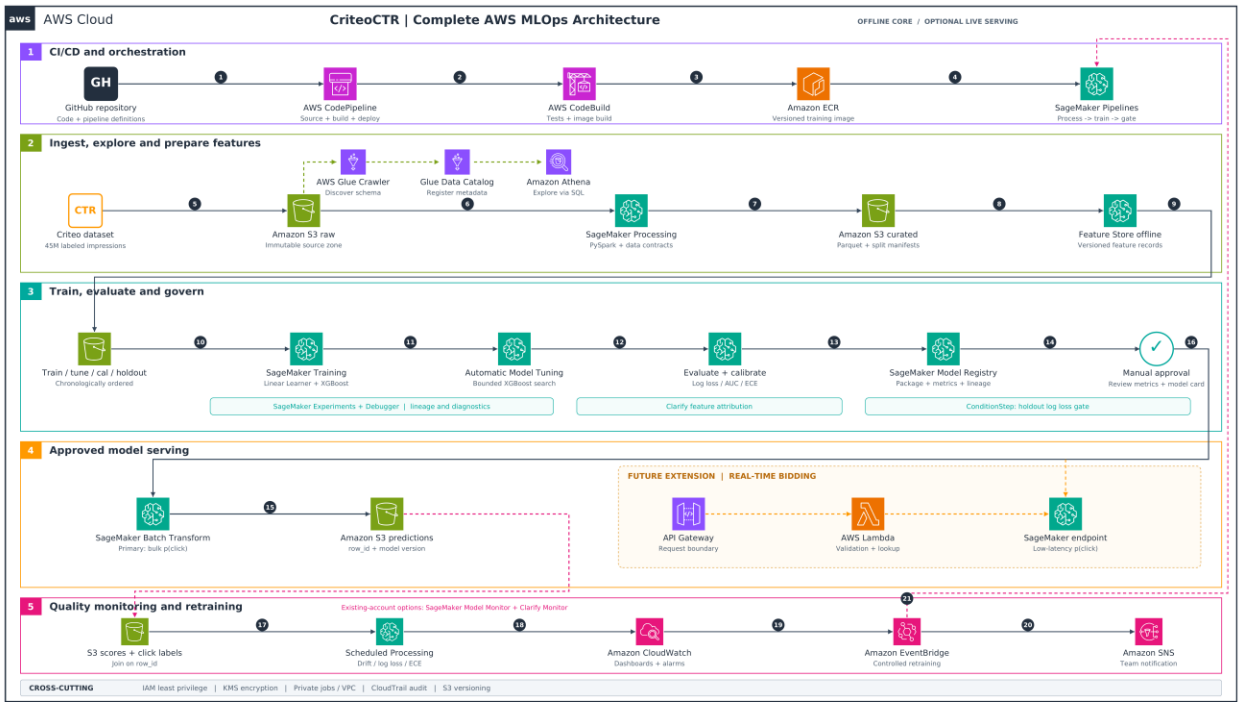


Figure 1. CriteoCTR complete AWS MLOps architecture — target design, with hand-offs numbered 1–21.

Figure 1 is the target design. In this project the following parts of it are scope rather than delivery, and the rest of this document describes what was built: CodePipeline and CodeBuild (GitHub Actions runs the tests; a runner holding AWS credentials is out of scope), PySpark processing at the full 45.8-million-row scale (processing runs with pandas on the sample, using identical transforms), an Athena catalogue written as explicit DDL rather than discovered by a Glue Crawler, Linear Learner (replaced by the two benchmarks), the API Gateway and Lambda front door for real-time bidding, and account-level KMS, VPC and CloudTrail controls that the course account does not let us configure. The weekly EventBridge retraining rule is created but left disabled until a first version is approved.

Pre-release testing:

- 115 automated tests that run in seconds with no AWS account: every transformation and metric against hand-computed values, the eight split checks, the leakage test, the pipeline contract, the step scripts end to end, and the data-acquisition module.
- A data-contract step inside the pipeline that fails the run on a malformed input file before any training is paid for.
- A pre-flight in the production notebook that runs every step script locally, proves the pipeline's preprocessing is byte-identical to the project library's, and asserts ten structural properties of the compiled pipeline definition before it is submitted.

- A four-condition quality gate on the held-out test split, followed by manual approval.
- A batch-versus-real-time parity check that both serving paths return the same score for the same row.

Post-release monitoring:

- Data quality, label-free and therefore first to fire: unseen-category rate per column, population stability index per numeric feature, missing rates and mean predicted CTR.
- Model quality once clicks arrive: log loss, AUC and calibration error against the metrics the deployed version was approved on.
- Infrastructure: invocations, 5xx errors, model and overhead latency, CPU and memory, on one CloudWatch dashboard with five alarms routed to SNS.

Data Sources:

What is your data source? The Criteo Display Advertising Challenge dataset from Kaggle (kaggle.com/c/criteo-display-ad-challenge), uploaded to our S3 landing folder as the original archive dac.tar.gz.

What is your data volume? train.txt holds 45,840,617 labelled impressions from seven consecutive days — about 11 GB uncompressed, inside a 4.3 GB archive — with one label, 13 integer and 26 hashed categorical features per row. The archive also contains test.txt, which has no labels; it is archived in the raw zone but never used, because it cannot be scored.

Why did you select this data set? It is an industry-scale CTR benchmark with an established evaluation standard, so results can be compared with published baselines rather than only with themselves. Its properties — scale, sparsity, extreme cardinality, missingness, hashed identifiers — reproduce the problems a real ad-bidding system faces, and the label is observed automatically from serving logs, so no labelling effort or annotator disagreement is involved.

Any risks (bias, sensitive features, etc.)? Three. Negatives were sub-sampled, so the base click rate is not a real-world rate and calibrated predictions from this model are not automatically live-traffic CTR. Every categorical feature is a hash, so we cannot see what it represents — a hashed user segment or location could proxy for a protected characteristic, and a fairness audit on protected classes is not possible. And the data contains only impressions the previous system chose to serve, so the model inherits that selection bias. These are covered in the Security Checklist below and stated in the model card attached to every registered version.

Data Engineering:

How will you store this data? Everything lives in Amazon S3 under s3://<bucket>/criteo-ctr/, in zones that separate immutable source data from derived artefacts. landing/ holds dac.tar.gz as uploaded. raw/ holds train.txt exactly as extracted; it is write-once, so any model version can be traced to the bytes it was built from. curated/ holds the chronological sample as Snappy Parquet partitioned by day_index. feature-store/ is the offline store for three feature groups. splits/, models/, predictions/ and monitoring/ hold the splits, model artefacts and evaluation reports, batch

output, and monitor baselines and reports. Athena tables over the raw file and the sample are defined with explicit DDL so the schema is reviewed in a pull request.

What data pre-processing do you need to do before you feed it into your ML system? Four stages. First, acquisition: the archive is extracted while it streams from S3, so it is never written to local disk, and each file is checked byte-for-byte against the size recorded in the archive before it is kept, so an interrupted transfer cannot leave a truncated train.txt that looks complete. Second, validation against a data contract — 40 columns, a binary label with both classes present, integer numerics, 8-character hexadecimal categoricals and null rates within bounds — which fails the run on violation. Third, sampling: every 45th row is kept, preserving chronological order across all seven days, and each row's position in the original file is stored as record_id; a head() sample would have covered part of day one and made a time-based split impossible. Fourth, conversion to Parquet with lower-case column names for Athena. Because the file has no timestamp, day_index and event_time are derived from row position and labelled as approximate wherever they appear.

Training Data:

How will you split your data into training, test and validation? Chronologically and contiguously: the earliest 40% of the sample is training, the next 10% validation (early stopping and tuning), the next 10% test (scored exactly once), and the most recent 40% is held back as simulated production traffic, with its labels withheld and released on a lag to emulate attribution delay. A random split would let impressions from the same session and campaign fall on both sides of the boundary, inflating every number and hiding the day-over-day drift the monitors exist to catch. Eight automated checks run before any split is written: row counts sum to the total, proportions are within tolerance, no record appears in two splits, splits are strictly ordered in time, no missing values survive, the encoder saw only training rows, both classes are present in training, and the feature columns match across splits.

Will you use any data labelling techniques? No. The label is observed automatically from Criteo's serving logs: 1 if the user clicked, 0 otherwise. No Ground Truth workflow is needed. Two label issues do need handling. Impressions near the end of a window may be labelled 0 only because the click has not been recorded yet, so model-quality monitoring joins predictions only to labels that have actually arrived and never treats a missing label as a non-click. And the classes are left as they are — neither resampled nor reweighted — because both change the base rate the model learns, and with it the calibration the bidding use case depends on.

Feature Engineering:

Which fields from your data will you use or exclude? All 39 predictors are used. Because they are anonymised there is no domain basis for excluding any of them; the unlabelled test.txt is excluded, and record_id, day_index and event_time are bookkeeping fields that never reach the model. The label is stored in its own Feature Store group so that nothing on the inference path can read it by accident.

Which fields will be combined or bucketed? The categorical columns. Each value is hashed into a fixed 2^20 space with a deterministic hash (Python's built-in hash() is salted per process and would produce different features in training and serving). Levels seen fewer than 10 times in the training window collapse into a shared rare bucket, and any level first seen after training maps to the same bucket with zero frequency, so new advertisers and publishers degrade gracefully instead of failing. Each categorical column then yields an encoded identifier and a frequency count.

What other data transformations will you apply to your data? A signed log transform, sign(x)·log1p(|x|), on the 13 count features — I2 takes negative values, so a plain log1p would return NaN — followed by imputation to zero and an explicit missing-indicator column for each, because missingness carries signal. The result is 78 model inputs. The design separates stateless transforms, which depend only on the row and are computed before splitting, from stateful ones — rare-level collapsing and frequency encoding — which learn from data and are therefore fitted on the training split alone and saved as an artefact for reuse at inference time. A unit test asserts that the encoder's counts equal counts from the training rows alone; a leaking model looks better, not worse, so nothing else would notice.

Model Training & Evaluation:

How will you train your model? As SageMaker Training Jobs on ml.m5.xlarge, using the built-in XGBoost 1.7-1 container on label-first CSV splits read from S3. SageMaker Debugger rules for loss-not-decreasing, overfitting and over-training watch each job, and SageMaker Experiments records the data version, image, hyperparameters and metrics of every run. In the production pipeline the same training step runs inside a SageMaker Pipeline with retry policies and 30-day caching, so an unchanged step is not re-paid.

What algorithm will you use? XGBoost with a binary:logistic objective: gradient-boosted trees handle skewed counts and high-cardinality frequency-encoded categoricals without scaling, and capture interactions without manual specification. It is compared against two benchmarks on the same split. The constant predictor is the instructive one: perfectly calibrated on average, it has a respectable log loss and an AUC of exactly 0.5, which is why the gate cannot rest on log loss alone. The two-feature logistic regression replaces the Linear Learner arm we originally planned.

What parameters will you use? The starting configuration is below. Automatic Model Tuning then runs a Bayesian search of 12 jobs, 3 in parallel, minimising validation:logloss over the ranges

shown, so that model selection, the reported metric and the pipeline gate all agree on what better means.

Hyperparameter	Starting value	Tuning range
objective / eval_metric	binary:logistic / logloss	—
max_depth	8	4 – 10
eta	0.1	0.02 – 0.3 (log scale)
min_child_weight	5	1 – 50
subsample	0.8	0.6 – 1.0
colsample_bytree	0.8	0.5 – 1.0
reg_lambda	1.0	0.1 – 10 (log scale)
num_round / early stopping	500 / 30 rounds	—
scale_pos_weight	1.0	fixed — see below

Class weighting is fixed at 1.0 deliberately. Our training notebook originally set scale_pos_weight to the negative-to-positive ratio, which is standard advice for imbalanced data. But weighting positives by k adds log k to every logit — the same effect as discarding negatives — so every probability is inflated. We measured it in SageMaker Studio (Figure 2): the reweighted model predicted a mean click rate of 0.345 against an observed 0.273, a 26% overbid on every impression, and its calibration error rose from 0.029 to 0.088 while its log loss also worsened. The production pipeline defaults to 1.0, and its calibration ceiling would reject a reweighted model. The 1M-row Criteo sample confirmed it on real data with a larger effect: weighting by the training ratio of 2.94 left AUC unchanged (0.771 against 0.772) but lifted the mean prediction to 0.412 against an observed 0.259, a 59% overbid, and raised calibration error from 0.005 to 0.153.

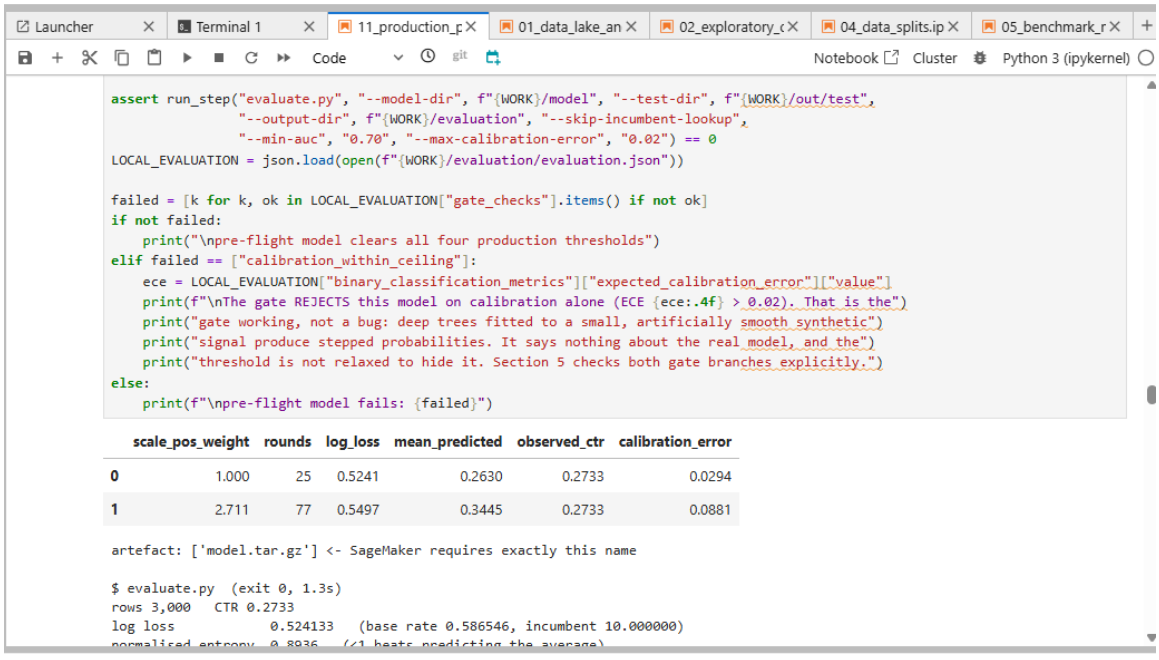


Figure 2. Effect of class weighting, measured in SageMaker Studio (notebook 11 pre-flight, synthetic Criteo-shaped data).

How will you evaluate your model? The test split is scored exactly once, by the same evaluation script the pipeline runs, so the number a reviewer reads is the number the gate reads. The evaluation reports log loss (primary), normalised entropy, ROC-AUC, PR-AUC, Brier score, expected calibration error with a decile calibration table, and precision, recall and F1 at a threshold equal to the base rate rather than 0.5 — at a low click rate a 0.5 cut-off predicts 'no click' for everything and reports flattering accuracy. Every metric is implemented once in numpy and unit-tested against hand-computed values. The output, evaluation.json, is the hinge of the system: the quality gate reads it, the Model Registry attaches it to the version as the metrics it was approved on, and model-quality monitoring compares live windows against it.

The gate has four conditions, all of which must hold: log loss at or below the base-rate log loss; log loss at or below that of the model currently approved in the registry, looked up at run time so a retrain must beat what is serving; AUC of at least 0.70; and ECE of at most 0.02. Figure 3 shows the first evaluation in SageMaker Studio. The candidate beat the base rate (log loss 0.524 against 0.587, normalised entropy 0.894), had no incumbent to beat, and cleared the AUC floor at 0.725 — but its calibration error of 0.029 exceeded the ceiling, so the gate rejected it, and the threshold was not relaxed to make it pass. These figures come from the pipeline's pre-flight on synthetic data shaped like Criteo.

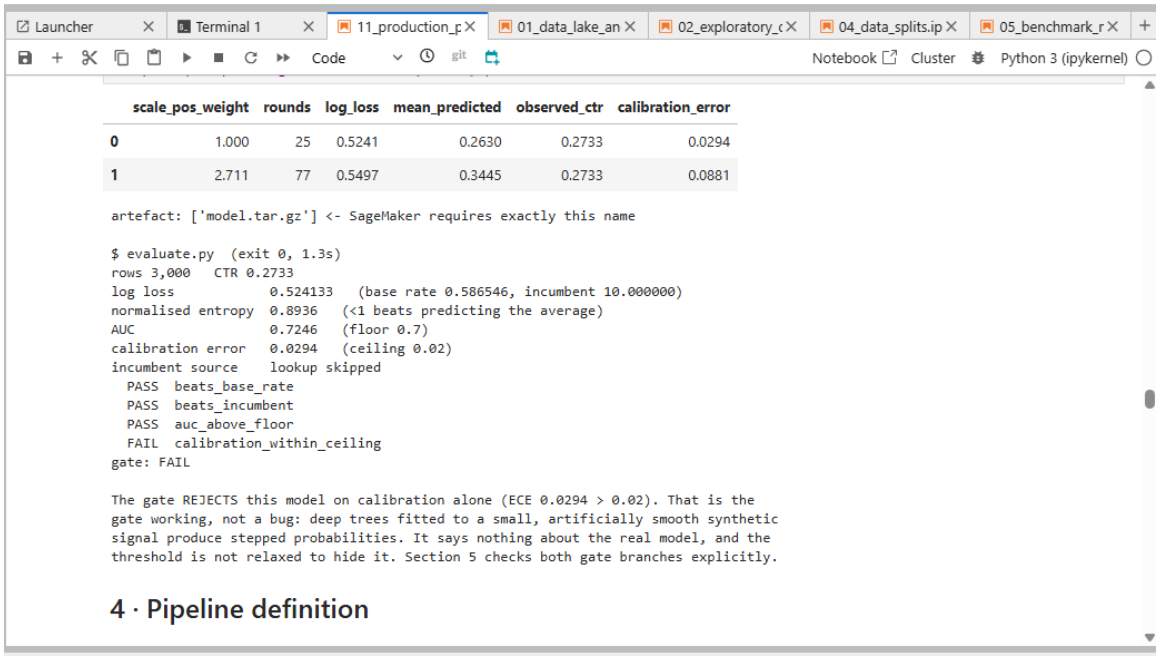


Figure 3. Pre-flight evaluation in SageMaker Studio: three conditions pass, calibration fails, and the gate rejects the model.

On real data the outcome is different. Trained on the first 400,000 rows of the 1M-row sample and scored once on 100,000 later test rows, with the same library and default hyperparameters, XGBoost passes all four conditions: log loss 0.4756 against a base rate of 0.5721 (normalised entropy 0.831), AUC 0.772 and ECE 0.005 — inside the monitor's 0.01 ceiling as well as the

gate's. Both benchmarks fail the AUC floor and the reweighted model fails on calibration. On the 400,000 held-back production rows it holds at log loss 0.4687 and AUC 0.774. This run was made with the project library outside SageMaker.

Test split, 100,000 rows	Global CTR	2-feature LR	XGBoost 1.0	XGBoost 2.94
Log loss	0.5721	0.5451	0.4756	0.5435
Normalised entropy	1.000	0.953	0.831	0.950
ROC-AUC	0.500	0.653	0.772	0.771
Expected calibration error	0.0051	0.0167	0.0048	0.1526
Mean predicted / observed	0.254 / 0.259	0.251 / 0.259	0.257 / 0.259	0.412 / 0.259
Gate verdict	Rejected (AUC)	Rejected (AUC)	Passes all four	Rejected (ECE)

Model Deployment:

What instance size will you use? ml.m5.large for both the real-time endpoint (one instance, 100% data capture) and Batch Transform, with ml.m5.xlarge for processing and training. Boosted-tree inference is CPU-bound with a small memory footprint, so GPU instances are not justified, and every instance type is a pipeline parameter that can be changed per execution without a code change. Instances are sized for this project's volume; a production deployment would confirm the type with a load test against a p99 latency budget.

Will your model function as a batch or real time model? Why? Both, from one registered artefact, with batch as the primary mode. Scoring the held-back production split is throughput-bound and embarrassingly parallel, so Batch Transform is the right tool and the source of the project's production numbers; keeping an endpoint alive to do it would pay for latency nobody needs. The real-time endpoint exists because the business case is real-time bidding and because the monitors need captured live traffic. Serving one model two ways creates a quiet risk — that the paths disagree — so a parity check scores identical rows through both and requires the same answer. Promotion is gated twice: the pipeline registers versions as PendingApproval, and deployment reads only the latest Approved package. Fronting the endpoint with API Gateway and Lambda, autoscaling, and blue/green release with shadow traffic are the future extension shown in Figure 1.

Model Monitoring:

How will you monitor your model? With a SageMaker Model Monitor model-quality schedule that runs hourly, joining captured predictions to ground-truth clicks and comparing log loss, AUC and calibration against the metrics in the approved version's own evaluation.json. The endpoint returns one probability per row for multi-row requests, so a custom record pre-processor fans each capture out into one record per scored row, keyed by inference ID and row offset. Calibration

has a threshold of its own because it degrades before ranking does and mis-prices bids while AUC still looks healthy. The thresholds are below.

Signal	Threshold	Why
Log loss vs approved baseline	+5% relative	Tolerates normal window-to-window noise
ROC-AUC vs baseline	−0.02 absolute	Operationally meaningful at bidding volume
Expected calibration error	0.01 ceiling (gate 0.02)	Directly mis-prices bids
Mean predicted CTR	±20% relative	Shift is visible before labels arrive

How will you monitor your infrastructure? A CloudWatch dashboard with eight widgets places model quality beside infrastructure health, so a drifting model and a saturated host — which need completely different responses — are never confused. Five alarms route to an SNS topic: p99 model latency above 100,000 microseconds (CloudWatch reports SageMaker latency in microseconds, and a threshold written as if it were milliseconds never fires), any 5xx error within five minutes, and CPU and memory above 80% for two consecutive periods. Email subscriptions must be confirmed before alerts are delivered, so confirmation is a setup step.

How will you monitor your data? With a Model Monitor data-quality schedule, also hourly, against statistics and constraints computed from the training features — in the production pipeline these are computed inside the pipeline and registered with the model version as drift-check baselines. The most informative signal for this dataset is the share of impressions whose categorical value the encoder never saw in training: it needs no labels and rises as soon as the advertiser or publisher mix changes. Its threshold is relative — 10 percentage points above the column's own training rate — because high-cardinality columns already send a large share of training rows to the rare bucket; our first absolute threshold fired against its own baseline. Numeric features are checked with the population stability index (0.10 warn, 0.25 alarm). Schema violations are caught at ingestion by the data contract. On the 1M-row sample, PSI between training and production rows is below 0.01 for twelve of the thirteen numeric features and 0.084 for I5, and each high-cardinality column's rare-bucket share is within two points of its training value, so the monitors would correctly stay quiet.

Model CI/CD:

What checkpoints will your CI/CD pipeline contain? The production SageMaker Pipeline (criteo-ctr-production, registered in our Studio domain) runs ten steps from the raw file. Its checkpoints are: the data-contract step, which fails the run on a malformed feed; a data-quality baseline step; training; evaluation; and the four-condition gate. On the pass branch the model is registered as PendingApproval with its metrics and drift baselines, then used to batch-score the production split; on the fail branch a Fail step ends the execution and names the threshold missed, and nothing is registered. Manual approval in the registry is the final checkpoint before deployment. Eighteen parameters cover input, instance types, every hyperparameter and both thresholds; job steps retry on throttling, service faults and capacity errors; step code is stored at a content-addressed S3 path so the code version is part of the model's lineage; and each

execution writes to its own output prefix. Both branches have been exercised: a run that passes and a run configured to fail.

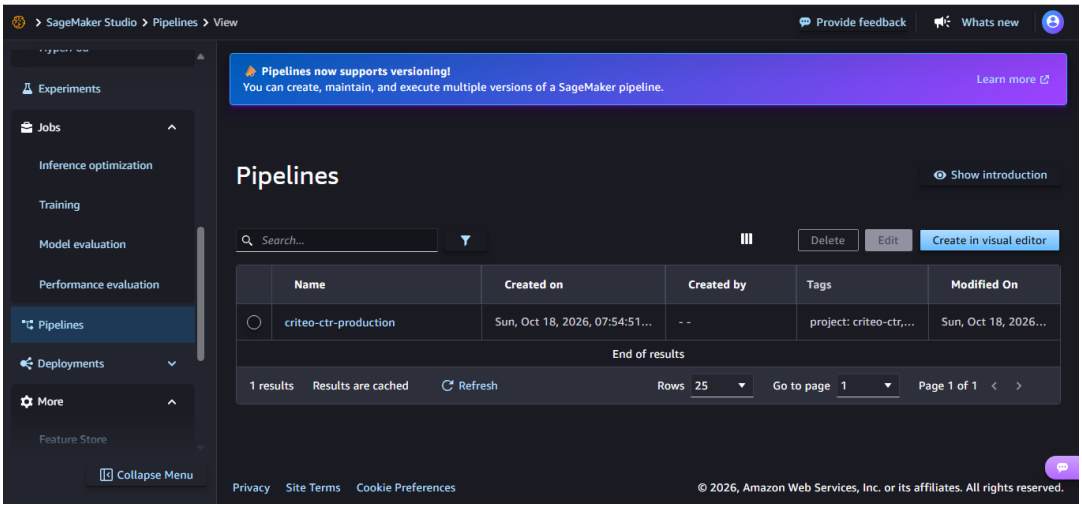


Figure 4. The criteo-ctr-production pipeline registered in SageMaker Studio.

What tests will your CI/CD pipeline contain? On every push, GitHub Actions runs the 115-test suite, the pipeline contract check, an end-to-end dry run of the whole system on synthetic data (about three seconds) and a notebook parse check, plus lint. The suite covers every transformation and metric against hand-computed values, the split checks and leakage test, the step scripts end to end including the model.tar.gz round trip, and data acquisition including truncated transfers. Before submission, the production notebook adds the byte-for-byte parity test between the pipeline's preprocessing and the library, and ten checks on the compiled pipeline definition. Inside the pipeline, the data contract and the quality gate are tests in their own right, and the serving parity check runs after deployment.

Security Checklist, Privacy and Other Risks:

Will this store or process Personal Health Information (PHI)? No. The dataset contains advertising impression logs only, with no health attributes.

Will this store or process Personal Identifiable Information (PII)? No direct PII. The 26 categorical features are hashed by Criteo before release and the 13 numeric features are anonymised counts. Some hashed columns may be stable pseudonymous identifiers, which data-protection law can treat as personal data, so the data is handled with the access controls we would apply to PII and no attempt is made to re-identify anyone or reverse the hashing.

Will user behavior be tracked and stored? Yes, in the sense that click behaviour is the label: the dataset is a record of whether users clicked ads, and the model predicts future clicks from past ones. Individual predictions are never stored against an identifiable person.

Will this store or process credit card information? No. No payment or financial data is present.

What S3 buckets will this application read from or write to? One bucket — the account's SageMaker default bucket unless CRITEO_BUCKET is set — under the prefix criteo-ctr/: landing/ (read), raw/ (write-once, then read-only), curated/, feature-store/, splits/, models/, predictions/ (including data capture), monitoring/, artifacts/, athena-results/ and prod-pipeline/ (per-execution outputs and content-addressed step code). Access is through the SageMaker execution role. KMS customer-managed keys, private VPC subnets with PrivateLink and CloudTrail auditing are the production controls in Figure 1; they are account-level settings outside this project's scope.

What data bias should be considered? Selection bias is the most fundamental: the data contains only impressions the previous system chose to serve, so the model never learns about inventory it was not shown, and acting on its scores narrows the distribution further unless some traffic is deliberately explored. Popularity bias favours high-volume advertisers with more examples. Temporal bias follows from a seven-day window that cannot represent seasonal or campaign cycles. And the sub-sampled negatives mean the base rate is not a real-world rate.

Will your model have potential for bias along sensitive features? Potentially, and it cannot be ruled out. No feature is labelled as a protected attribute, but a hashed user segment, location or publisher can act as a proxy, and because the features are hashed we cannot test for it. The documented harm in advertising is differential ad delivery — housing, credit or job ads reaching groups unevenly with no protected attribute in the model. SageMaker Clarify reports which features drive predictions, and the model card states plainly that a fairness audit on protected classes is not possible on this data, rather than reporting a check that would mean nothing.

Are there ethical concerns with the data or business problem? Yes. Behavioural advertising rests on large-scale tracking that users rarely meaningfully consent to; anonymisation reduces but does not remove that concern. Optimising purely for clicks rewards attention-grabbing creative over relevant advertising — a production system should optimise advertiser return, with click probability as an input. And regulated ad categories carry legal as well as ethical weight. Within the project we make no re-identification attempts, document limitations in the model card, report feature attribution, and require a human approval before any deployment.

Future Enhancements:

- **Deep interaction models.** Hashing and frequency encoding discard how categorical levels relate to one another; learned embeddings recover it, and published results on this dataset show meaningful log loss gains for DeepFM and wide-and-deep models over boosted trees. This is the highest-value next step and needs GPU training instances.
- **A streaming feature pipeline with an online store.** Recency features — recent click rate per advertiser, frequency-capping counters — are among the strongest CTR signals and impossible in a batch design. Kinesis for impression events, a streaming aggregation job and the Feature Store online store would make them available at inference time with sub-millisecond lookup.

- **Online evaluation with exploration.** Offline log loss is a proxy for business value. An A/B framework measuring realised revenue per thousand impressions, with a contextual-bandit component serving a small share of uncertain impressions, would measure true impact and break the selection-bias feedback loop.
- **Closing the automation and serving gap.** Enable the weekly EventBridge retraining rule once a version is approved, start the pipeline automatically on a monitor violation, build containers through CodePipeline from a protected branch, and add API Gateway, Lambda, autoscaling and blue/green release with canary traffic for real-time bidding.
- **Full-scale processing and threshold reconciliation.** Run processing in PySpark over the full 45.8-million-row file to prove the scalability the sample can only assert, re-tune with the class weight fixed at 1.0, and reconcile the gate's calibration ceiling (0.02) with the monitor's (0.01) on real data.